PCET's Pimpri Chinchwad University
Learn | Grow | Achieve

Name :- Jaykishan Saharan
Course :- B.Tech CSE-SY (core)
Division :- D
Roll No :- 26

DSA THEORY ASSIGNMENT 1

08/10

Q1. Define data structure. Compare different types of data structure with example.

Sol. A data structure is a way of organizing, managing and storing data so that it can be accessed and modified efficiently.
It defines the relationship between data elements and the operations that can be performed on them.

Types of Data Structure

Data structures are broadly classified into two categories:

1 Primitive Data structures
- These are the basic data types provided by a programming language.
- Examples :- Integer (int) → stores numbers (eg. int x = 10;)
- Float (float) → stores decimal values (eg. float pi = 3.14;)
- Characters (char) → stores single characters (eg. char c = 'A';)
- Boolean (bool) → stores true or false.

Limitation : They cannot handle complex relationship between data.

2. Non-Primitive Data Structures
These are more advanced and used to store and manage collections of data.
They are divided into linear and Non-linear data structures.

(A) Linear Data Structures
Data is arranged in a sequence; each element is connected to the next.

1. Array
- Fixed-size collection of elements of the same type.
- Example: int arr[5] = {10, 20, 30, 40, 50};
- Access is fast using index.
- Limitation: Size is fixed.

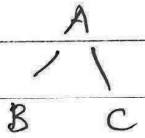
2. Linked List
- Collection of nodes where each node contains data + pointer to next node.
- Example: 10 → 20 → 30 → NULL
- Flexible size but slower access compared to arrays.

3. Stack
- Follows LIFO (Last in First Out) principle.
- Example: Browser history (last visited page is the first to go back).
- Operations: Push, Pop, Peek.

4. Queue
- Follows FIFO (First in First Out) principle.
- Example: People standing in a ticket line.
- Variants: Circular Queue, Deque, Priority Queue.

(B) Non-Linear Data Structures
Data is not stored sequentially, elements can have hierarchial or interconnected relationships.

1. Tree
- Hierarchial structure with root, parent, child nodes.
- Example:



- Used in file systems, databases.

2. Graph
- Collection of nodes (vertices) and edges (connections).
- Example: Social networks (users as nodes, friendships as edges).
- Types: Directed, Undirected, Weighted.

Q2. Compare
- Linear & Non-linear data structure
- Static & Dynamic data structure

with example of each.

- Linear and Non-linear data structure

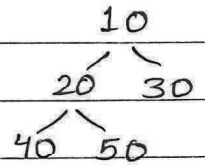
1. Linear Data Structure
   - Definition: In a linear data structure, elements are arranged sequentially (one after another).
   - Traversal: Can be traversed in a single run (start to end).
   - Memory usage: Elements are stored in contiguous or logically sequential memory locations.
   - Examples: Array, Linked List, Stack, Queue

Example: Array
int arr[5] = {10, 20, 30, 40, 50};
- Stored sequentially in memory.
- Traversed from index 0 to 4.

Example: Stack (LIFO)
Push → [10, 20, 30] → Pop → removes 30 first.

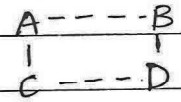
2. Non-Linear Data Structure
   - Definition: In a non-linear data structure, elements are not sequential; they are connected hierarchially or graph-like.
   - Traversal: Requires multiple runs or special techniques (DFS, BFS).
   - Memory usage: Elements are stored in non-contiguous memory locations.
   - Example: Trees, Graphs, Heaps, Hash Tables

Example: Tree



- Parent-child relationship.
- Traversal can be Inorder, Preorder, Postorder.

Example: Graph



- Nodes connected in multiple directions.
- Traversal via BFS / DFS.

- Static and Dynamic Data Structures

1. Static Data Structure
   - Definition: A data structure whose size is fixed at compile-time.
   - Memory Allocation: Done before execution (compile-time)
   - Flexibility: Cannot grow or shrink during program execution.
   - Efficiency: Access is fast (because of contiguous memory), but memory may be wasted if unused.
   - Examples: Array, Static Stack

   Example: Array (C++)

   ```
   int arr[5] = {10, 20, 30, 40, 50};
   ```

   - Memory allocated once.
   - Cannot add 6th element.

2. Dynamic Data Structure
- Definition: A data structure whose size is flexible and can change at runtime.
- Memory allocation: Done at runtime using dynamic memory (heap).
- Flexibility: Can grow or shrink as needed.
- Efficiency: No memory wastage, but more complex to manage.
- Examples: Linked list, Dynamic stack / Queue.

Example: Linked list (C++)

```
struct Node {
    int data;
    Node* next;
};
Node* head = NULL;
```

- New nodes can be created/deleted anytime.
- No need to define a fixed size.

Q3. What is asymptotic notation? Explain Big O notation with example.

Sol. Asymptotic notation is a way to describe the efficiency of an algorithm in terms of:
- Time Complexity (how fast it runs)
- Space Complexity (how much memory it uses)

It helps us analyze how the algorithm behaves as the input size (n) grows very large (tends to infinity). Instead of focusing on exact running time, we express growth mathematically.

The three most common notations are:

1. Big O (O) → worst case upper bound
2. Big Ω (Ω) → Best case lower bound
3. Big Θ (Θ) → Tight bound (both upper & lower)

Big O Notation (O)

Big O describes the upper bound of an algorithm - how fast its runtime can grow at most as input size increases.

- It tells us the worst-case growth rate.

Example 1: Simple Loop

```
for (int i=0; i<n; i++) {
    cout << i;
}
```

- This loop runs n times.
- Time complexity = O(n)

Example 2: Nested Loop

```
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++) {
        cout << i << ", " << j;
    }
}
```

- Inner loop runs n times for each outer loop.
- Total operations = $n \times n = n^2$
- Time complexity = $O(n^2)$

Example 3 : Mixed Operations

```
for (int i=0; i<n; i++) {    // O(n)
    cout << i;
}
for (int j=0; j<n; j++) {    // O(n)
    cout << j;
}
```

- First loop = O(n)
- Second loop = O(n)
- Total = O(n+n) = O(2n) → simplify to O(n)

Q4. Define the terms Data structure. Abstract Data Type (ADT) & Flowchart with example of each.

Sol: Data Structure

- Definition: A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.
- Types:
- Linear: Array, Stack, Queue, Linked list
- Non-linear: Tree, Graph, Hash Table

Example: Array

```
int arr[5] = {10, 20, 30, 40, 50};
```

- Stores 5 integers in a sequential way.

2. Abstract Data Type (ADT)

- Definition: An ADT is a mathematical model for a data structure, which defines the behaviour (what operations can be performed) but does not specify the implementation details (how it is done).

Examples of ADTs:

- ~~Definition: An~~ A Stack (ADT): operations → Push, Pop, Peek
- Queue (ADT): Operations → Enqueue, Dequeue

Example: Stack ADT

- Operations defined:
- push (x) → Insert element x
- pop () → Remove top element
- Doesn't specify whether stack is implemented using an array or linked list.

3. Flowchart

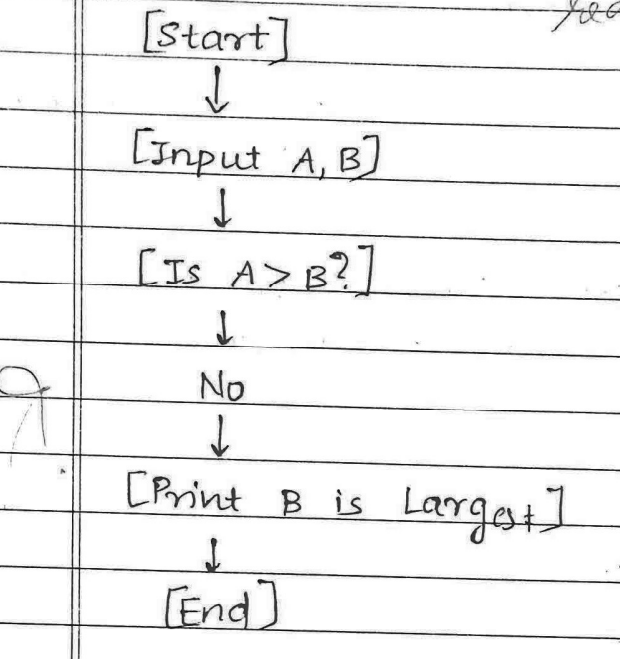
Definition: A flowchart is a diagrammatic representation of an algorithm, process, or program using symbols (like rectangle, diamonds, ovals).

It shows the flow of control step by step.

Common Symbols:

- Ovals → Start/End
- Rectangle → Process / Instruction
- Diamond → Decision (Yes / No)
- Arrow → Flow direction

Example: Flowchart to find largest of two numbers redraw:



Q5. Construct a flowchart to compute the sum of n integers along with its pseudocode.

Sol. Pseudocode: compute Sum of n integers

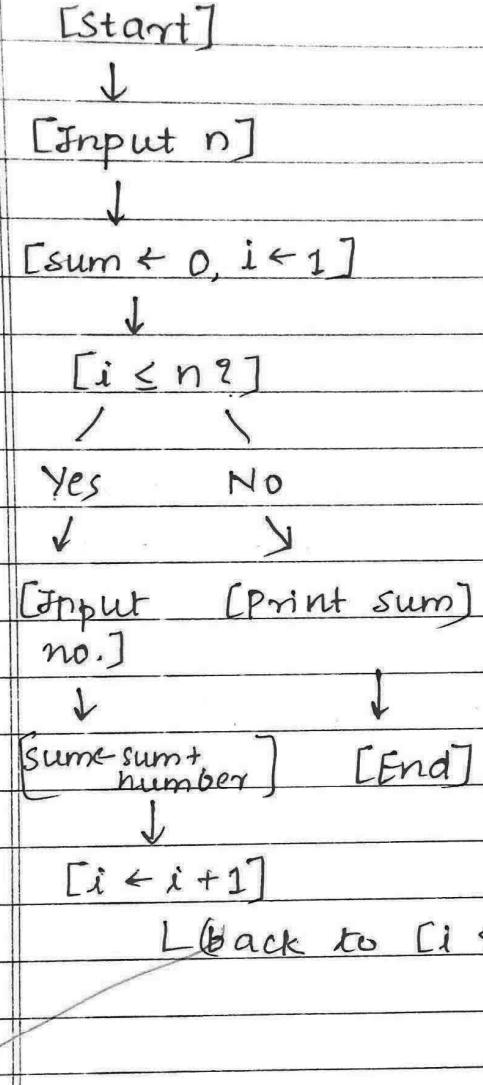
```
Start
Input n
sum ← 0
i ← 1

while i ≤ n do
    Input number
    sum ← sum + number
    i ← i + 1
End while

Print sum
Stop
```

PCET's Pimpri Chinchwad University
Learn | Grow | Achieve

Flowchart: Compute Sum of n Integers



Q6. Examine given snippets to find the time complexity.

i)
```
for (i=0; i<n; i++)
    {
    for (j=0; j<n; j++)
        {
        cout << i << j;
        }
}
```

Step 1: Analyze loops
- Outer loop runs from $i=0$ to $i<n$ → execute n times
- Inner loop runs from $j=0$ to $j<n$ → execute runtime for each iteration of outer loop.

Step 2: Total operations
- Inner cout statement executes $n * n = n^2$ time

Step 3: Time Complexity
- $O(n^2)$ - Quadradic time

Ans - $O(n^2)$

(ii)
```
int main()
    {
    int a=10, b=20, c;
    c = a+b
    cout << "\n Addition =" << c;
    }
```

Ste

Step 1: Analyse operations

- int a = 10, b = 20, c; → 8 assignments → O(1)
- c = a+b; → 1 addition → O(1)
- cout statement → 1 output → O(1)

Step 2 : Total operations

- All statements are executed once regardless of input size.

Step 3 : Time complexity

- O(1) → constant time.

Answer O(1)